Department of Computer Science and Engineering

Islamic University of Technology (IUT)

CSE 4851

Design Pattern Assignment 1

Personalized Itinerary Planner

Implementation of
Strategy, Decorator, Factory, and Template Method Patterns

Prepared by: Rumman Adib
Student ID: 210041131
Section: 1
Submission Date: 05 June, 2026

Source Code Repository

https://github.com/Rumman023/Travel-Planner

Contents

1. Scenario Definition

1.1. Overview

The scenario is a **personalized travel itinerary planning system** for a commercial company or for personal use. The system must handle travelers with very different needs: backpackers, luxury tourists, adventure seekers and families, etc.

Each traveler needs a custom itinerary. The system must:

- Build itineraries in a structured, step-by-step algorithmic way.
- Use different recommendation algorithms at runtime depending on the traveler's goals.
- Create the right type of itinerary object without exposing the creation logic.
- Allow optional add-on services (insurance, local guides, meal plans) to be added flexibly at runtime.

No single pattern can satisfy all four requirements. This is why four patterns are combined and used together in this particular scenario. s

1.2. Why This Scenario Fits All Four Patterns

Pattern	Role in This Scenario
Template Method	Defines the fixed five-step workflow for building any itinerary.
Strategy	Swaps the recommendation algorithm (AI, Budget, Quality, Culture) at runtime.
Factory Method	Creates the correct itinerary subclass based on traveler type.
Decorator	Wraps a built itinerary with optional add-on services.

The four patterns form a natural pipeline. The Factory creates the object. The Template Method runs the build steps. The Strategy is injected into those steps to fill in the recommendations. Finally after the itinerary is built, Decorators adds on extra services.

2. Pattern Implementations

2.1. Template Method Pattern

Files: `BaseItinerary.java`, `BackpackerItinerary.java`, `FamilyItinerary.java`, `PremiumItinerary.java`, `ExtremeSportsItinerary.java`

The Template Method pattern defines a fixed algorithm skeleton in a base class. Subclasses can override specific steps without changing the overall structure.

In this system, `BaseItinerary` defines `buildItinerary()` as a `final` method. It calls five steps in order:

1. `gatherPreferences()` : reads the traveler profile (fixed)
2. `selectDestinations()` : chooses destinations (overridable hook)
3. `bookActivities()` : books activities (overridable hook)

4. `arrangeTransport()` : arranges transport (fixed)
5. `generateItineraryReport()` : prints the final report as a PDF (fixed)

Steps 1, 4, and 5 are identical for all traveler types. Steps 2 and 3 are hook methods and subclasses override them to add type-specific behavior.

For example, `ExtremeSportsItinerary` ignores the strategy output entirely in Step 2 and hardcodes adrenaline-rated destinations. `FamilyItinerary` uses the strategy destinations as-is but adds child-friendly filters in Step 3.

```
public final void buildItinerary() {
    gatherPreferences();     // Step 1 (fixed)
    selectDestinations();    // Step 2 (overridable hook)
    bookActivities();        // Step 3 (overridable hook)
    arrangeTransport();      // Step 4 (fixed)
    generateItineraryReport(); // Step 5 (fixed)
}
```

Listing 1: Template Method skeleton in BaseItinerary.java

2.2. Strategy Pattern

Files: `RecommendationStrategy.java`, `AIRecommendationStrategy.java`, `BudgetStrategy.java`, `QualityStrategy.java`, `LocalCultureStrategy.java`

The Strategy pattern defines a family of interchangeable algorithms behind a common interface. The algorithm can be swapped at runtime without changing the class that uses it.

`RecommendationStrategy` is the interface. It declares three methods: `recommendDestinations()`, `recommendActivities()` and `getStrategyName()`. Four concrete strategies implement it.

- **AIRecommendationStrategy**: simulates an ML model, so here no real AI models/api is used. It analyzes the traveler's past trips and matches interests to predict destinations. All of this is hard coded.
- **BudgetStrategy**: selects the cheapest destinations and free or low-cost activities.
- **QualityStrategy**: scores destinations by their price-to-quality ratio and picks verified top-rated options.
- **LocalCultureStrategy**: avoids tourist hotspots and recommends immersive, off-the-beaten-path cultural experiences.

The strategy is injected into `BaseItinerary` through the constructor. This means the same factory can produce itineraries with different recommendation behaviors simply by passing a different strategy object.

The system includes a strategy swap demo. The same backpacker profile is run through `BudgetStrategy` and then `QualityStrategy`. The itinerary type stays the same, but the destination output changes. This demonstrates correct runtime substitution.

```
// Same factory, same profile, only strategy differs
BaseItinerary budgetRun = budgetFactory.createItinerary(profile, new
    BudgetStrategy());
budgetRun.buildItinerary();

```

```
BaseItinerary qualityRun = budgetFactory.createItinerary(profile, new
    QualityStrategy());
qualityRun.buildItinerary();
```

Listing 2: Strategy swap at runtime in TravelPlannerApp.java

2.3. Factory Method Pattern

Files: `ItineraryFactory.java`, `ItineraryFactoryProvider.java`

The Factory Method pattern delegates object creation to subclasses. The client asks for an object by type and receives the correct concrete instance without knowing which class was instantiated.

`ItineraryFactory` is the abstract factory interface with one method: `createItinerary()`. Four concrete factories implement it:

- **BudgetFactory**: creates `BackpackerItinerary`
- **LuxuryFactory**: creates `PremiumItinerary`
- **AdventureFactory**: creates `ExtremeSportsItinerary`
- **FamilyTravelFactory**: creates `FamilyItinerary`

`ItineraryFactoryProvider` acts as the entry point. It takes a string like `"adventure"` and returns the correct factory using a switch expression. The client in `TravelPlannerApp` never directly instantiates any itinerary subclass. This keeps object creation centralized and easy to extend.

```
public static ItineraryFactory getFactory(String travelerType) {
    return switch (travelerType.toLowerCase()) {
        case "backpacker", "budget"  -> new BudgetFactory();
        case "luxury", "premium"     -> new LuxuryFactory();
        case "adventure", "extreme"  -> new AdventureFactory();
        case "family"                -> new FamilyTravelFactory();
        default -> throw new IllegalArgumentException("Unknown type: "
           + travelerType);
    };
}
```

Listing 3: Factory provider using a switch expression

2.4. Decorator Pattern

Files: `ItineraryDecorator.java`, `TravelInsuranceDecorator.java`, `LocalGuideDecorator.java`, `MealPlanDecorator.java`, `AirportTransferDecorator.java`, `PhotoPackageDecorator.java`

The Decorator pattern wrap an existing object to add new behavior. The wrapper implements the same interface as the wrapped object, so it can be used transparently.

`ItineraryDecorator` is the abstract decorator. It extends `BaseItinerary` and holds a reference to a `wrappedItinerary`. All delegate calls (destinations, activities, transport, profile) go to the wrapped object.

Five concrete decorators add optional services:

- **TravelInsuranceDecorator**: calculates cost as 5% of the traveler's budget.
- **LocalGuideDecorator**: charges per destination city at a daily rate.
- **MealPlanDecorator**: charges per day for breakfast and dinner.
- **AirportTransferDecorator**: chages per airport transfer based on the number of destinations.
- **PhotoPackageDecorator**: adds a fixed cost for a professional photographer over two days.

Decorators are not stacked on each other here. Instead, they each wrap the same base itinerary and are collected in a list. This is a deliberate design choice: it allows the adds on costs to be tallied independently without the complexity of chained delegation.

3. Pattern Interaction

The four patterns work together as a pipeline. The flow for each traveler scenario is:

1. **Factory**: The client calls `ItineraryFactoryProvider.getFactory("adventure")` and receives an `AdventureFactory`.
2. **Factory**: The factory's `createItinerary()` is called with a `TravelerProfile` and a `RecommendationStrategy`. It returns an `ExtremeSportsItinerary`.
3. **Template Method**: `buildItinerary()` is called. The five-step workflow runs. The strategy is used in Steps 2 and 3, but `ExtremeSportsItinerary` overrides those steps to apply an extreme-sport filter first.
4. **Decorator**: After the itinerary is built, the client wraps it in one or more decorator objects (e.g., `TravelInsuranceDecorator`, `LocalGuideDecorator`). Each decorator adds its service details and cost on top of the built itinerary without modifying the original object.

The strategy is the only pattern that crosses the boundary between two others. It is created outside the factory, passed into the factory, and then used inside the template method. This is intentional. It means the factory does not decide the recommendation algorithm, and the template method does not know which algorithm it is using. Responsibility is cleanly separated.

4. Test Scenarios

4.1. Scenario 1: Miraj (Adventure Traveler)

Miraj is a solo traveler with a budget of $2,500 USD (converted to BDT) for 14 days. His interests include rock climbing, skydiving, surfing, etc. His past trips include Asia and South America.

- **Factory:** `AdventureFactory` creates an `ExtremeSportsItinerary`.
- **Strategy:** `AIRecommendationStrategy` is injected. It detects past Asia travel and recommends Bhutan and Kyrgyzstan.
- **Template:** The extreme filter overrides Step 2 to lock in Queenstown, Moab, Interlaken, and Chamonix.

- **Decorators:** Travel insurance (5% of budget), local guide service (per city), and a photography package is applied.

4.2. Scenario 2: Rakib Family (Family Travelers)

The Rakib family is a group of four with a budget of $6,000 USD for 10 days. Their interests include culture, history, food, and kid-friendly activities.

- **Factory:** `FamilyTravelFactory` creates a `FamilyItinerary`.
- **Strategy:** `LocalCultureStrategy` recommends off-the-beaten-path cultural destinations like Oaxaca and Luang Prabang.
- **Template:** Step 3 appends family-specific activities (wildlife sanctuary, kids' cooking class) on top of the strategy output.
- **Decorators:** A meal plan (per day) and airport transfers (per destination) are applied.

4.3. Scenario 3: Strategy Swap Demo (Jamil, a Backpacker)

Jamil has a budget of $800 USD for 7 days. The same `BudgetFactory` is used twice: once with `BudgetStrategy` and once with `QualityStrategy`. The itinerary type stays identical. The destination list changes completely. This demonstrates that the strategy is fully decoupled from the itinerary structure.

5. AI Usage in the assignment

The library to print out the output in PDF, 'i text pdf 5' was integrated in the project with the help of AI. Also to cleanly format of the output in PDF (which is a tedious task to do manually), AI was used.

6. Conclusion

This particular scenario shows how four design patterns can be combined, and also a necessary process in order to meet the strict requirements. Each pattern solves a different problem.Together, they produce a system that is open to extension but closed to modifcation. New traveler types, new strategies and new add-on services can be added independently without changing the existing code.